Patch-Aware Protein Design: Project Proposal

Members: Kirti Hargunani (kharguna), Ashwath Raghuram (ashwathr), Vani Nigam (vnigam)

1. Biomedical problem and technical challenges

The problem. Protein therapeutics fail in development largely through aggregation, specifically at the surface. Aggregation occurs due to clusters of hydrophobic residues on the solvent-accessible surface, which can be undetectable by models that score based on hydrophobic content overall. Aggregation is a key concern for designed protein therapeutics, particularly at high formulation concentrations (e.g., 100–150 mg/mL). Miniprotein binders have limited core volume to bury hydrophobic residues, potentially leaving aggregation-prone patches exposed on their surfaces. Aggregation can reduce manufacturing yield and increase the risk of anti-drug antibody responses, which may compromise therapeutic efficacy.

Technical challenges.

- **Decoding-order blindness.** ProteinMPNN decodes residues autoregressively under a random permutation, so when any position is assigned, roughly half of its spatial neighbours are still undetermined. Each residue's marginal is well calibrated while the joint configuration over a local surface neighbourhood is left unconstrained.
- **No credit assignment.** Property-aware methods score one scalar per sequence of 50–500 residues, which carries no information about which residues caused the deficit.
- **Proxy exploitation.** Sequence-based solubility predictors are composition-dominated. MoMPNN's (Hou et al, 2026) surface net charge reaches −14.161 per 100 residues against ProteinMPNN's +0.892 : roughly 3× past the subset-trained baseline with no wet-lab validation reported.
- **Objective conflict is spatial.** A binder's interface must present exposed hydrophobics; the rest of the surface must not. A single global trade-off weight cannot express this.
- **Label scarcity at interfaces.** Measured binding ΔΔG exists for ~7,000 mutations (SKEMPI 2.0), against ~10^5–10^6 extractable surface patches.

2. Existing state-of-art methods on this problem and code availability

Method	Contribution	Availability
ProteinMPNN (2022)	Inverse folding; 52.4% recovery vs 32.9% Rosetta	open

LigandMPNN (2025)	Ligand-aware context; near-ligand recovery 50.4% → 63.3%	open
SolubleMPNN (2024)	Retrained on transmembrane-excluded subset	checkpoint
HyperMPNN (2024)	Retrained on hyperthermophiles	open
ProtAlign / MoMPNN (ICLR 2026)	Multi-objective DPO over property predictors	checkpoints only, no training code
ResiDPO / EnhancedMPNN (2025)	Residue-level DPO; binder success 7.07% → 16.07%	partial
MaSIF (2020, 2023)	Patch-level contrastive surface fingerprints	open
RFdiffusion(2023) / BindCraft (2025)	Backbone generation; BindCraft 10–100% success, mean 46.3%	open

3. Direction going forward

- **Patch extraction and encoding.** Surface and interface regions are extracted from the backbone, and each patch is encoded from geometry plus residue identity.
- **Contrastive training on matched pairs.** We permute amino acid assignments within a patch (identical composition, different arrangement) so composition-counting solutions carry no gradient and any discrimination above chance is attributable to arrangement.
- **Property heads.** A shared trunk with per-property heads, each defined by its own pair-construction rule: within-patch permutation (aggregation), and SKEMPI mutation pairs (interface).
- **Region-specific scores.** Because scores are computed per patch, objective weights can vary by region within one protein and one forward pass. Aggregation penalties are applied depending on region. This differentiates our program from the whole-sequence scalar reward methods applied by previous extensions of ProteinMPNN.
- **Two optimization routes: Reranking** stock ProteinMPNN samples and **fine-tuning** with preferred/rejected pairs.

4. Evaluation of Performance

- **General Evaluation:** Test whether scores agree with held-out labels or measurements. Use ranking accuracy for ordered pairs, AUROC, precision–recall metrics for classification, and correlation or prediction error.

- **Comparison:** composition-matched negatives versus MaSIF-style cross-protein negatives, under an identical encoder to quantify how much established patch-learning performance is attributable to residue composition.
- **Negative controls**
- **Fold fidelity:** TM-score
- **Design-stage:** binder success rate on ProtAlign's five targets under their criteria (binder pLDDT inter-chain PAE, Cα RMSD) against ProteinMPNN, SolubleMPNN and MoMPNN.
- **Check useful properties and structure:** literature review

5. Team and workload

Workstream	Scope
A. Data and candidate generation	Select backbones and target regions, run ProteinMPNN, extract surface and interface patches, build the rerank baseline
B. Patch scorer	Patch encoder, contrastive training on matched permutations and labelled pairs, property heads (aggregation, stability, interface), region-specific scores
C. Optimization and evaluation	Reward-ranked fine-tuning, metrics for properties, fold fidelity (TM-score) and diversity

Deliverables:

Deliverable	Target (a day early)
Mid report	Nov 5 (due Nov 6)
Final report	Dec 5 (due Dec 6)
Poster	Dec 5 (due Dec 6)

6. Timeline

Date	Milestone
Oct 12	Environment set up, target proteins chosen, ProteinMPNN running
Oct 19	Candidate sequences generated, patch extraction working, evaluation metrics defined
Oct 26	Scorer v1 trained (encoder and contrastive loss)

Nov 2	Property heads and region-specific scores done, rerank baseline results, mid report drafted
Nov 5	**Mid report submitted (due Nov 6)**
Nov 6 – Nov 16	Explore the fine tuning frameworks possible for the property heads which are ready
Nov 17 – Nov 25	Work on the selected framework for finetuning
Nov 28 – Dec 1	Full evaluation and ablations: rerank vs fine tune
Dec 2 – Dec 4	Final report and poster drafted, cross-reviewed
Dec 5	**Final report and poster submitted (due Dec 6)**

7. Infrastructure and compute

Resource	Estimated requirement	Use
CPU	8–16 cores per preprocessing job	Structure filtering, patch extraction, dataset splitting and evaluation
System memory	64 GB RAM	Processing structures and loading patch datasets
GPU	One NVIDIA A100 or H100 GPU with 24–48 GB VRAM per training job	Patch scorer training, ProteinMPNN generation and fine-tuning
Structure prediction	Access to a GPU with 40–80 GB VRAM if needed for larger complexes	Fold fidelity and computational binder evaluation
Storage	Approximately 300–500 GB	Structures, cached patch features, model weights, checkpoints and evaluation outputs
Software	Python, PyTorch, a graph learning library, structural analysis tools and an AlphaFold-based evaluation pipeline	Model development and reproducible analysis

Our initial compute estimate assumes approximately 100,000–300,000 training patches, several scorer experiments and comparison of fine-tuning methods. Structure prediction will be restricted to a predefined evaluation set.

Stage	Estimated computation
Data preparation and patch extraction	300–600 CPU core-hours
Candidate generation, scoring and reranking	10–20 GPU-hours
Patch scorer training, including controls and repeat runs	40–80 GPU-hours
ProteinMPNN fine-tuning experiments	50–100 GPU-hours
Structure prediction and final design evaluation	100–200 GPU-hours
Expected GPU total	**200–400 GPU-hours**
Requested allocation, including contingency	**500 GPU-hours and 1,000 CPU core-hours**

The GPU estimate is a rough budget for an A100-class accelerator.